

**CURSO PROFISSIONAL TÉCNICO DE GESTÃO E
PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS**

PROVA DE APTIDÃO PROFISSIONAL

imoral

Fábio Vitoriano

Ciclo 2023-2026

Agradecimentos

Este projeto nunca teria sido realizado sem o apoio, colaboração e incentivo de diversas pessoas, a quem quero expressar a minha mais sincera gratidão.

Em primeiro lugar, quero agradecer ao meu colega Arthur de Mello Mattos, que foi o meu colega de PAP e que está comigo desde o momento em que esta ideia surgiu.

Quero agradecer ao professor José Augusto da Rocha Gonçalves, que foi o nosso maior orientador durante todo o processo de desenvolvimento da PAP e que sempre disponibilizou a sua ajuda e atenção quando necessário.

Gostaria de agradecer também à nossa Diretora de Curso, Maria Luísa Parente, que me ajudou durante o desenvolvimento do projeto e sempre esteve lá por nós ao longo deste curso.

Agradeço também aos meus amigos Carlos Eduardo Freire e David Coelho, que sempre me deram esperança e estenderam a sua mão para me ajudar durante todo o desenvolvimento do projeto.

Por último, gostaria de expressar a minha gratidão a todos os professores, colegas, familiares e amigos que me apoiaram e tornaram este projeto possível.

Índice

Agradecimentos	2
Índice.....	2
1. Resumo	3
2. Palavras Chave	4
3. Introdução.....	4
3.1 Contexto	5
3.2 Motivação	5
3.3 Objetivos.....	5
3.3.1 Objetivo Geral.....	6
3.3.2 Objetivos Específicos	6
4. Análise de Requisitos	7
4.1 Requisitos funcionais	7
4.2 Requisitos não funcionais	8
4.3 Restrições	8
5. Metodologia.....	8
5.1 Metodologia de desenvolvimento	9
5.2 Ferramentas e Tecnologias Utilizadas	9
5.2.1 Aplicações e Ferramentas	9
5.2.2 Linguagens e tecnologias.....	10
5.3 Cronograma	10
6. Desenvolvimento	13
6.1 Arquitetura de Sistema.....	14
6.1.1 Funcionamento da API	14
Autenticação e Segurança (Login e JWT)	17
6.1.2 Modelo ER	17
6.2 Design da Interface.....	22
6.3 Implementação.....	22
6.3.1 Base de Dados	22
6.3.2 Aplicação Android.....	23
6.3.3 Comunicação entre a App e a API: Exemplo Prático	23
6.4 Testes e validações	24
7. Resultados.....	26
7.1 Funcionalidades Desenvolvidas	27
7.2 Benefícios Esperados	27
7.3 Limitações	27

8. Conclusão.....	29
8.1 Síntese do Projeto	29
8.2 Reflexão Crítica	29
8.3 Melhorias Futuras.....	29
9. Referencias	29

1. Resumo

No âmbito da finalização do curso Técnico de Gestão e Programação de Sistemas Informáticos (TGPSI), é-nos requisitada uma Prova de Aptidão Profissional (PAP) como projeto final. Para este projeto, eu e o meu colega Arthur de Mello Mattos desenvolvemos a Imoral, uma marca de roupa e espaço de comunidade online. A ideia deste projeto surge da necessidade de mais marcas independentes que apostem na sustentabilidade em Portugal.

A Imoral serve como loja online de roupa com foco em produtos personalizáveis. O cliente, antes de comprar a peça, pode alterá-la adicionando elementos como correntes e tecidos, de forma a tornar aquele produto à sua imagem. Escolhemos implementar esta funcionalidade por acreditarmos que a moda devia ser mais sobre expressão pessoal e unicidade. A nossa marca transmite uma imagem de choque, com cores escuras, imagens mais cruas e materiais mais rasgados nas roupas, tendo sido escolhido o nome do projeto já com essa ideia em mente.

Para além da loja, também desenvolvemos um espaço de comunidade, algo similar a uma rede social ou a um fórum. Esta parte do projeto serve para reunir as pessoas interessadas em moda alternativa em Portugal. Queremos que os nossos clientes e interessados em moda tenham um espaço para comunicar sobre interesses, divulgar eventos e partilhar fotografias. Com este espaço será também possível colaborar com outras marcas independentes e, dessa forma, criar uma relação mais forte dentro de um mercado que acreditamos estar ainda muito fragmentado.

2. Palavras Chave

Moda, Comunidade, choque, sustentabilidade, D.I.Y (Do It Yourself).

3. Introdução

3.1 Contexto

Hoje em dia, o setor da moda é controlado por grandes empresas que colocam o lucro acima da qualidade e da sustentabilidade dos seus produtos. Estas empresas têm controlo absoluto sobre o mercado da moda, podendo definir o que as pessoas vestem e os preços praticados. A Imoral visa combater isto ao promover a sustentabilidade e a expressão pessoal.

3.2 Motivação

O que me motivou a realizar este projeto foi a minha paixão pela moda e o interesse em criar as minhas próprias roupas, aliados à perceção da falta de marcas independentes de estilo alternativo em Portugal. Devido a esta escassez de marcas e também ao pouco destaque das que já existem, pretendo que a Imoral traga uma plataforma e um sentido de comunidade para as pessoas que partilham os mesmos interesses em moda alternativa e procuram um espaço para falar sobre eles.

Com este projeto pretendo também promover a compra de roupa de forma sustentável, sem alimentar empresas multimilionárias que pouco se importam com o meio ambiente e com a qualidade das peças que vendem.

3.3 Objetivos

3.3.1 Objetivo Geral

Criar uma aplicação que permita vender as roupas da minha marca, incluindo produtos que podem ser personalizados pelo cliente antes da compra. Nessa mesma aplicação, existirá uma secção que funcionará como um fórum, especificamente para interessados em moda que procuram um espaço para partilhar opiniões, fotografias e ideias.

3.3.2 Objetivos Específicos

- Criar um painel administrativo que nos permita, a nós e a futuros administradores, ter controlo sobre o que é publicado na comunidade, verificar o estado das encomendas, verificar o stock da loja e também consultar as estatísticas, tanto da loja como da comunidade.
- Cada utilizador terá o seu carrinho de compras e a sua lista de desejos dentro da loja.
- Criar um fórum com a possibilidade de fazer publicações com texto e imagens, e de comentar nas publicações de outros utilizadores.
- Dispor de uma página de início de sessão e de registo, com opção de recuperação de palavra-passe através do email.

3.4 Estrutura do relatório

Este relatório está dividido em 8 tópicos: no começo inclui um resumo do projeto, que pretende resumir o projeto em poucas palavras para que qualquer leitor consiga entender do que se trata sem precisar de ler o relatório todo; as palavras-chave que definem o projeto; uma introdução que apresenta do que se trata este projeto e o que é pretendido com ele; a análise de requisitos, que mostra quais os requisitos que o sistema deve ter; a metodologia, que pretende apresentar como foi a realização do projeto no âmbito de métodos de trabalho e de ferramentas utilizadas; o desenvolvimento, que explica como o projeto e as suas ideias foram postas em prática; logo após é abordado os resultados que o desenvolvimento teve e, por fim, temos a conclusão e reflexões finais sobre o projeto.

4. Análise de Requisitos

4.1 Requisitos funcionais

- O sistema deve permitir ao utilizador adicionar produtos ao seu carrinho, escolhendo o tamanho e a quantidade que pretende comprar.
- O utilizador deve ter a possibilidade de ver os produtos que tem no carrinho e poder alterar os tamanhos e as quantidades.
- O sistema deve fornecer um histórico de compras ao utilizador.
- Para além do carrinho, o utilizador também deve ter a sua lista de desejos, na qual pode adicionar produtos em que está interessado.
- O sistema deve permitir ao utilizador fazer upload de imagens para personalizar o seu perfil e partilhar imagens no fórum.
- Os utilizadores devem conseguir comentar nas publicações uns dos outros e conseguir responder a comentários específicos nas mesmas.

- O sistema deve permitir aos utilizadores ver os perfis uns dos outros, com todas as fotografias e publicações incluídas.
- O utilizador deve poder personalizar certos produtos antes de os comprar.
- O sistema deve permitir-nos, enquanto administradores, banir ou suspender temporariamente contas através do painel administrativo.
- Todos os dados de encomendas e pedidos realizados devem ser armazenados e visíveis no painel de administração.

4.2 Requisitos não funcionais

- O painel administrativo, a aplicação e o site promocional devem partilhar a mesma base de dados.
- A base de dados deve ser atualizada o mais rapidamente possível após o utilizador realizar alguma ação relevante.
- As passwords de todos os utilizadores devem ser encriptadas quando adicionadas na base de dados.

4.3 Restrições

- O sistema não permite publicar mais do que uma fotografia por publicação.
- A função de loja e de fórum foram desenvolvidas apenas para a aplicação Android.
- A funcionalidade de selecionar quais os produtos do carrinho que serão comprados não foi desenvolvida.

5. Metodologia

5.1 Metodologia de desenvolvimento

Uma vez que este projeto foi realizado em dupla, foi totalmente necessário dividir tarefas e rever mutuamente o trabalho, de modo a assegurar que tudo decorria de acordo com as expectativas de ambos.

A nossa forma de trabalho baseou-se na metodologia de Desenvolvimento Ágil de Software. Na prática, isto significa que seguimos um ciclo iterativo no qual planeámos as tarefas antes de as realizar, criámos layouts ou protótipos que solidificaram a ideia para o software, desenvolvemos as funcionalidades e testámos rigorosamente o que foi programado, antes de reiniciarmos o ciclo para cada etapa do projeto.

5.2 Ferramentas e Tecnologias Utilizadas

5.2.1 Aplicações e Ferramentas

- **Laragon** - Ambiente de desenvolvimento web local que permite criar, testar e gerir aplicações web diretamente no computador.
- **Ngrok** - Ferramenta de tunelamento que fornece uma URL pública para um servidor local em execução no computador.
- **Visual Studio Code** - Editor de código utilizado para programar o site promocional, o painel administrativo e a Api em php.
- **Figma** - Editor visual que permite criar *mockups* interativos para aplicações moveis.
- **Android Studio** - Ambiente de desenvolvimento integrado (IDE) dedicado a criação de aplicativos destinados a dispositivos *Android*.
- **PhpMyAdmin** - Aplicação web que funciona como uma interface gráfica para gerir bancos de dados MySQL e MariaDB.

- **BlueJ** - Ambiente de Desenvolvimento Integrado (IDE) projetado especificamente para a linguagem de programação Java.
- **GitHub** - Plataforma de hospedagem de código-fonte e arquivos, utilizada para armazenar e partilhar código.
- **PostMan** - Plataforma colaborativa e ambiente de desenvolvimento utilizada para criar, testar, documentar e gerir APIs.

5.2.2 Linguagens e tecnologias

- **Java e XML** - Utilizadas para o desenvolvimento da aplicação móvel destinada a dispositivos *android*.
- **PHP** - Utilizada para o desenvolvimento da API e para o *backend* do painel de administração permitindo gerir os dados da base de dados e realizar chamadas HTTP.
- **SQL** - Utilizada para o desenvolvimento da base de dados do projeto e das suas *queries*.
- **HTML, CSS e JavaScript** - Utilizadas para o desenvolvimento do site promocional e do painel de administração (BackOffice).

5.3 Cronograma

Tarefas	Data Inicio	Data Final
Pre-Plano	10/09	22/11
Criação do Modelo ER	25/9	15/12
Plano	1/11	20/12
Criação da Base de Dados em sql	1/12	1/03
Design dos Layouts da APP	1/01	3/03
Preparação da apresentação intermediária	10/02	5/03
Programação da API em php	1/05	15/06
Programação da App em Java	02/01	24/06
Realização do relatório de trabalho	1/05	21/06
Preparação da apresentação final	10/06	26/06

Pré-Plano e Plano: Antes de começarmos a desenvolver o projeto, foi-nos pedido um Pré-Plano e um Plano, onde definíamos as nossas expectativas para o projeto, como surgiu a ideia e quais as tecnologias e linguagens que esperávamos usar.

Criação do Modelo ER: A criação de um Modelo ER foi também uma das etapas essenciais no início do projeto, dando-nos a oportunidade de ter um modelo que representasse a estrutura da base de dados e de todo o projeto. Apesar de termos finalizado a criação do Modelo ER no início do projeto, este foi sendo modificado ao longo do desenvolvimento.

Criação da Base de Dados em SQL: Esta etapa tornou-se bastante fácil e intuitiva graças ao Modelo ER que já tínhamos criado, mas, tal como o modelo, a base de dados também passou por alterações ao longo do projeto.

Design dos Layouts da APP: Esta etapa serviu para criar os layouts que iríamos usar na aplicação. Para isso, foi usado o Figma, que nos permite criar layouts interativos de aplicações. Foi uma das etapas mais simples, mas também uma das que mais contribuiu para o projeto.

Preparação da Apresentação Intermédia: Nesta etapa foi-nos pedido que fizéssemos uma apresentação onde mostrássemos o que já tínhamos no projeto e o que pretendíamos ainda desenvolver, de forma a prepararmo-nos para a apresentação final da PAP.

Programação da API em php: Programar a API foi, no início, uma etapa bastante complicada para mim, mas, depois de começar a perceber melhor o seu funcionamento, tornou-se razoavelmente fácil e revelou-se bastante necessária para o projeto.

Programação da APP em java: Programar a APP foi, de longe, a parte mais complicada de todo o projeto. Algumas funcionalidades que queria implementar acabaram por ser cortadas, e a integração da API na aplicação também foi, no início, complicada para mim.

Realização do Relatório de Trabalho: Para a realização do relatório, reuni toda a informação relevante do projeto e organizei-a em secções, tendo em conta a estrutura que nos foi indicada. O relatório serve para que alguém que não acompanhou o desenvolvimento do projeto possa entender as várias etapas que levaram à sua execução, bem como o funcionamento dos sistemas.

Preparar a Apresentação Final: A última etapa do projeto foi a preparação da apresentação da PAP, que consistiu na criação do PowerPoint a apresentar e em bastante treino e preparação para possíveis perguntas.

6. Desenvolvimento

6.1 Arquitetura de Sistema

O sistema Imoral foi desenvolvido com uma arquitetura cliente-servidor composta por três aplicações cliente e uma API, que serve de ponte entre estas aplicações e a base de dados:

- **Aplicação Móvel (Java)** – Permite aos utilizadores criar conta, visitar a loja, realizar compras, interagir com publicações de outros utilizadores e fazer as suas próprias publicações.
- **Site promocional (HTML, CSS, JavaScript)** – Permite aos utilizadores criar uma conta e subscrever a newsletter da marca.
- **Painel Administrativo (HTML, CSS, JavaScript)** – Permite aos administradores ter controlo sobre o fórum e sobre a loja.
- **API (PHP)** – Funciona como o núcleo do sistema, processando os pedidos enviados pelas três aplicações cliente, comunicando com a base de dados e devolvendo as respostas em formato JSON. É também responsável pela autenticação dos utilizadores, através de tokens JWT, controlando o acesso aos dados consoante as permissões de cada um.

6.1.1 Funcionamento da API

Para demonstrar o funcionamento da API, vamos usar como exemplo prático os pedidos relacionados com os posts do fórum, percorrendo as várias camadas envolvidas no processamento de um pedido: ligação à base de dados, modelo de dados, DAO, controller e rotas.

Ligação à Base de Dados

O ficheiro Database.php é responsável por estabelecer a ligação à base de dados. Esta classe guarda as credenciais de acesso e disponibiliza o método connect(), que cria e devolve uma ligação PDO sempre que é chamado por um DAO.

```
<?php

class Database
{
    private $host = 'localhost';
    private $db_name = 'imo_system';
    private $username = 'root';
    private $password = '1234';

    public function connect()
    {
        $conn = null;

        try {
            $conn = new PDO(
                "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
                $this->username,
                $this->password
            );
            $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch (PDOException $e) {
            echo "Erro de conexão: " . $e->getMessage();
        }

        return $conn;
    }
}
```

Fig.1 - Classe Database.php

Modelo de Dados

A classe Post.php representa o modelo de dados de um post, funcionando de forma semelhante a um model em Java. Tem métodos get e set para cada atributo, bem como um método que devolve todos os dados do objeto num array, facilitando a sua conversão para JSON.

```
mydev.amoral.com > app > models > Post.php
3 class Post {
33     public function getId(): int { return $this->id; }
34     public function setId(int $id): void { $this->id = $id; }
35
36     public function getIdUtilizador(): int { return $this->id_utilizador; }
37     public function setIdUtilizador(int $id_utilizador): void { $this->id_utilizador = $id_utilizador; }
38
39     public function getDtPostagem(): string { return $this->dt_postagem; }
40     public function setDtPostagem(string $dt_postagem): void { $this->dt_postagem = $dt_postagem; }
41
42     public function getTextoPost(): string { return $this->texto_post; }
43     public function setTextoPost(string $texto_post): void { $this->texto_post = $texto_post; }
44
45     public function getLikeCount(): int { return $this->like_count; }
46     public function setLikeCount(int $like_count): void { $this->like_count = $like_count; }
47
48     public function getUsername(): ?string { return $this->username; }
49     public function getImage(): ?string { return $this->image; }
50     public function getCommentCount(): int { return $this->comment_count; }
51
52     public function toArray(): array {
53         return [
54             'id' => $this->id,
55             'id_utilizador' => $this->id_utilizador,
56             'dt_postagem' => $this->dt_postagem,
57             'texto_post' => $this->texto_post,
58             'like_count' => $this->like_count,
59             'username' => $this->username,
60             'image' => $this->image,
61             'comment_count' => $this->comment_count
62         ];
63     }
}
```

Fig.2 - Classe Post

DAO (Data Access Object)

No PostDAO, são utilizadas as classes Post.php e Database.php para criar as funções que permitem interagir diretamente com os dados da base de dados. Como exemplo, a função `getAllPosts` vai à base de dados buscar todos os posts, juntando dados do utilizador (nome e imagem) e a contagem de likes e comentários de cada post, devolvendo no final um array de objetos Post.

```
mydev.imoral.com > app > dao > PostDAO.php
6 class PostDAO {
60     public function getAllPosts(): array {
61         $sql = "
62             SELECT
63             p.id,
64             p.id_utilizador,
65             u.username,
66             u.image,
67             p.dt_postagem,
68             p.texto_post,
69             COUNT(pl.id) as like_count,
70             COUNT(c.id) as comment_count
71         FROM posts as p
72         INNER JOIN post_likes as pl ON pl.id_post = p.id
73         INNER JOIN utilizadores as u ON p.id_utilizador = u.id
74         LEFT JOIN comentarios as c ON c.id_post = p.id
75         GROUP BY p.id
76         ";
77
78         $stmt = $this->conn->prepare($sql);
79         $stmt->execute();
80
81         $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
82
83         $posts = [];
84
85         foreach($rows as $row) {
86             $posts[] = $this->rowToPost($row);
87         }
88
89         return $posts;
90     }
```

Fig.3 - Function `getAllPosts()` da Classe PostDAO

Controller

No PostController, o PostDAO é utilizado para criar funções que respondem diretamente aos pedidos HTTP. Na função `getAllPosts`, os dados devolvidos pelo DAO são colocados numa lista e enviados numa resposta em formato JSON, que indica se a operação foi realizada com sucesso ou não, incluindo a mensagem de erro caso algo tenha falhado.

```
mydev.imoral.com > app > controllers > PostController.php
7  {
14  public function findPostById($postId) {
37      };
38
39      Utils::jsonResponse($dataResponse, 401);
40  }
41  }
42
43  public function getAllPosts() {
44  try {
45      $posts = (new PostDAO())->getAllPosts();
46
47      $dataResponse = [
48          'success' => true,
49          'message' => "Operação realizada com sucesso",
50          'data' => [
51              'posts' => array_map(fn($post) => $post->toArray(), $posts)
52          ]
53      ];
54
55      Utils::jsonResponse($dataResponse, 200);
56  } catch(Exception $e) {
57      $dataResponse = [
58          'success' => false,
59          'message' => $e->getMessage(),
60          'data' => []
61      ];
62  };
63
64      Utils::jsonResponse($dataResponse, 400);
65  }
66  }
```

Fig.4 - Função `getAllPosts()` da Classe PostController

Rotas HTTP

O ficheiro `index.php` é responsável por definir as rotas HTTP, associando cada URL e método (GET, POST, PUT, DELETE) à função correspondente nos controllers. Por exemplo, um pedido GET ao endereço `/posts` é encaminhado para a função `getAllPosts` do PostController, que devolve a lista de posts em formato JSON.

```
mydev.imoral.com > public > api > index.php
104 else if ($uri === "/pedido_produtos" && $method === 'POST') {
108 }
109
110 else if ($uri === "/posts" && $method === 'GET') {
111     AuthMiddlewareApi::check();
112     (new PostController())->getAllPosts();
113 }
114
115 else if ($uri === "/posts" && $method === 'POST') {
116     AuthMiddlewareApi::check();
117     (new PostController())->createPost();
118 }
119
120
121 else if (preg_match('/^\/posts\/(\d+)\/', $uri, $matches) && $method === 'DELETE') {
122     AuthMiddlewareApi::check();
123     $postId = $matches[1];
124     (new PostController())->deletePost($postId);
125 }
126
127
128 else if (preg_match('/^\/posts\/(\d+)\/', $uri, $matches) && $method === 'PUT') {
129     AuthMiddlewareApi::check();
130     $postId = $matches[1];
131     (new PostController())->updatePost($postId);
132 }
133
134
135 else if (preg_match('#^\/posts\/(\d+)#', $uri, $matches) && $method === 'GET') {
136     $postId = $matches[1];
137     AuthMiddlewareApi::check();
138 }
```

Fig.5 - Rotas dos pedidos do PostController no ficheiro Index.php da API

Autenticação e Segurança (Login e JWT)

O acesso à API é protegido através de autenticação por JWT (JSON Web Token). Quando o utilizador faz login, a API valida as credenciais introduzidas com os dados guardados na base de dados e, caso estejam corretas, gera um token JWT assinado, que contém informação como o id do utilizador e a data de expiração. Este token é devolvido à aplicação, que o guarda localmente para autenticar os pedidos seguintes.

Sempre que a aplicação faz um pedido a um endpoint protegido, o token é enviado no header Authorization, e a API verifica a sua validade e tempo de expiração antes de processar o pedido. Caso o token seja inválido ou tenha expirado, a API recusa o pedido e devolve um erro, obrigando o utilizador a autenticar-se novamente.

```
mydev.imoral.com > app > controllers > AuthController.php
13 class AuthController{
189 public function loginApi() {
190     try {
191         $body = json_decode(file_get_contents('php://input'), true);
192
193         $email = trim($body['email'] ?? $_POST['email'] ?? '');
194         $password = trim($body['password'] ?? $_POST['password'] ?? '');
195
196         // Se não houver email ou password, mostrar erro
197         // é preciso lançar exceção para o index.php apanhar e mostrar o erro via flash message
198         if (empty($email) || empty($password)) {
199             throw new Exception("Email e password são obrigatórios");
200         }
201
202         $user = (new UserDAO())->findByEmail($email);
203
204         if (! $user || ! password_verify($password, $user->getPassword())) {
205             throw new Exception("Email ou password errados");
206         }
207
208         $carrinho = (new CarrinhoDAO())->findCarrinhoById($user->getId());
209
210         $data = [
211             'id' => $user->getId(),
212             'role' => $user->isAdmin()
213         ];
214
215         $payload = jwtConfig::getConfig($data);
216
217         $jwt = JWT::encode($payload, jwtConfig::getSignature(), "HS256");
218     }
```

Fig.6 - Função LoginApi na classe AuthController

6.1.2 Modelo ER

O Modelo ER (Modelo Entidade-Relacionamento) é um modelo concetual de dados usado para projetar e representar a estrutura de uma base de dados de forma visual e abstrata. A criação de um Modelo ER foi uma das primeiras tarefas que nos foram propostas e ajudou-nos imensamente a mapear as informações que seriam necessárias para ter uma base de dados sólida, que nos permita consistência tanto na loja como no fórum da nossa marca.

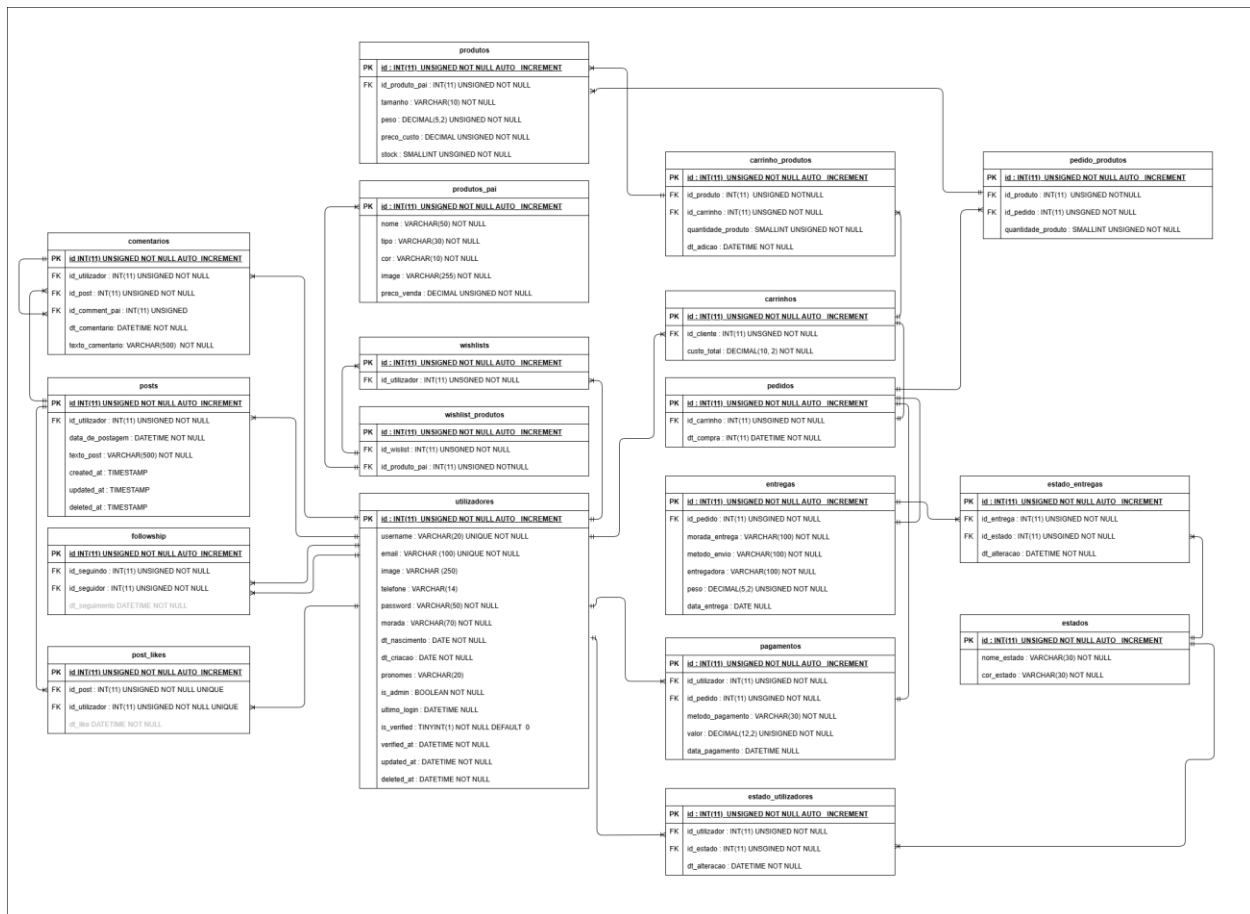


Fig.7 - Modelo ER da Base de dados do projeto

Todas as tabelas do modelo ER têm como PK (Primary Key) o respetivo id, atribuído automaticamente a cada novo registo inserido na tabela. Para estabelecer ligações entre tabelas, são criados atributos FK (Foreign Key), que referenciam valores existentes em atributos de outras tabelas.

- **Utilizadores**

Esta é a tabela central da base de dados. Armazena todos os dados do utilizador e é usada tanto na loja como no fórum.

- **Followship**

Esta tabela serve para gerir que contas seguem outras contas. Quando um utilizador segue outro, é criado um insert com o id do utilizador que seguiu na FK id_seguidor e com o id da conta seguida por esse utilizador na FK id_seguido. Esta tabela também armazena a data em que o utilizador seguiu o outro, para que no futuro possamos implementar um sistema de notificações.

- **Posts**

Esta tabela armazena os dados de um post feito por um utilizador no fórum. Tem uma FK id_utilizador para guardar o id do criador do post, e guarda também o texto e a data em que foi publicado.

- **Post_likes**

Esta tabela funciona de forma semelhante à tabela Followship e serve para gerir quais utilizadores deram like num post, tendo a FK id_utilizador com o id de quem deu like no post e a FK id_post com o id do post que recebeu o like.

- **Comentários**

Esta tabela serve para armazenar os comentários realizados nos posts. Guarda o id do autor do comentário na FK id_utilizador, o id do post na FK id_post, e a data e hora em que o comentário foi feito no atributo dt_comentario. Caso o comentário seja uma resposta a outro comentário, o id desse comentário original é guardado no atributo FK id_comentario_pai, que estabelece uma relação da tabela consigo própria (auto-relacionamento, usado para representar respostas em cadeia).

- **Produtos_pai**

Esta é também uma das tabelas centrais da base de dados. Nela são armazenados os detalhes principais dos produtos, como por exemplo o preço de venda, o nome, o tipo, a cor e a imagem que irá aparecer na app e no site.

- **Produtos**

Esta tabela é auxiliar à tabela Produtos_pai. Nela são armazenados cada tamanho de cada produto-pai e os dados que não foram guardados na tabela Produtos_pai por variarem consoante o tamanho. Esta tabela tem uma FK id_produto_pai que guarda o id do produto-pai, um atributo que define o tamanho da peça, um que armazena o stock disponível desse tamanho, e o atributo preco_custo, que guarda o custo de produção de cada peça.

- **Wishlist**

A tabela Wishlist serve para associar cada utilizador à sua wishlist, tendo como único atributo, além do seu id, a FK id_utilizador.

- **Wishlist_produtos**

Esta tabela é auxiliar à tabela Wishlist e serve para armazenar os produtos (Produtos_pai) inseridos numa wishlist, tendo a FK id_wishlist, que armazena o id da wishlist, e a FK id_produto_pai, que armazena o id do produto adicionado. Esta é também uma das tabelas centrais da base de dados. Nela são armazenados os detalhes principais dos produtos, como por exemplo o preço de venda, o nome, o tipo, a cor e a imagem que irá aparecer na app e no site.

- **Carrinho**

A tabela Carrinho funciona de forma muito semelhante à tabela Wishlist. Serve para associar cada utilizador ao seu carrinho e, além da FK id_utilizador, conta também com o atributo custo_total, que armazena o valor total do carrinho.

- **Carrinho_produtos**

Esta tabela funciona de forma muito semelhante à tabela Wishlist_produtos, sendo uma tabela auxiliar à tabela Carrinho. Guarda o id do produto e o id do carrinho quando um produto é adicionado, e a quantidade de peças desse produto que o utilizador pretende comprar é armazenada no atributo quantidade.

- **Pedidos**

A tabela Pedidos serve para gerir as compras realizadas. Quando um utilizador tem produtos no carrinho e confirma uma compra, é criado um insert nesta tabela, armazenando o id do carrinho na FK id_carrinho e a data em que a compra foi feita no atributo dt_compra.

- **Pagamentos**

Esta tabela serve para gerir os pagamentos feitos pelos utilizadores durante uma compra. Armazena na FK id_utilizador o id do utilizador que realizou o pagamento, na FK id_pedido o id do pedido ao qual o pagamento corresponde, no atributo metodo_pagamento o método usado para pagar (por exemplo, MB WAY ou PayPal), no atributo valor o montante pago, e no atributo data_pagamento a data e hora em que o pagamento foi realizado.

- **Pedido_produtos**

Esta tabela funciona de forma idêntica à tabela Carrinho_produtos, tendo como propósito guardar os produtos comprados num pedido. O id do pedido é armazenado na FK id_pedido, replicando os mesmos registos que o carrinho tinha antes da compra. Desta forma, é possível limpar o carrinho assim que a compra é confirmada, transferindo os produtos adquiridos para o pedido.

- **Entregas**

Esta tabela serve para armazenar os dados de uma entrega. Guarda o id do pedido correspondente na FK id_pedido, a morada para onde os produtos serão enviados no atributo morada_entrega, o método de envio no atributo metodo_envio, a transportadora (por exemplo, CTT ou FedEx) no atributo entregadora, o peso da encomenda no atributo peso, e a data em que for entregue no atributo data_entrega.

- **Estados**

Esta tabela serve para gerir os estados que as contas ou encomendas podem ter, como por exemplo "Entrega cancelada" ou "Conta suspensa". Tem os atributos nome_estado e cor_estado, que guardam o nome e a cor do estado a exibir no painel e na app.

- **Estados_entregas**

Esta tabela é auxiliar à tabela Estados e serve para associar uma entrega ao seu respetivo estado, tendo a FK id_estado, que armazena o id do estado, a FK id_entrega, que armazena o id da entrega, e o atributo data_alteracao, que define quando o estado foi alterado. Através desta tabela, é possível ao utilizador acompanhar o estado da encomenda e o histórico de estados pelos quais passou.

- **Estados_Utilizadores**

Esta tabela é auxiliar à tabela Estados e serve para associar a conta de um utilizador ao seu respetivo estado, tendo a FK id_estado, que armazena o id do estado, a FK id_utilizador, que armazena o id do utilizador, e o atributo data_alteracao, que define quando o estado foi alterado. Esta tabela ajuda a gerir as contas no painel de administração, permitindo verificar de forma fácil os estados das contas dos utilizadores.

6.2 Design da Interface

Antes de começar a programar a aplicação, tive como prioridade criar *mockups* de como seria a sua interface. Para isto, usei o Figma, que permite criar *mockups* interativos para aplicações móveis. Esta ferramenta permitiu-nos, de forma simples e eficiente, colocar em prática a ideia que tínhamos em mente e realizar escolhas sobre a interface da nossa aplicação que a tornaram mais simples e intuitiva.

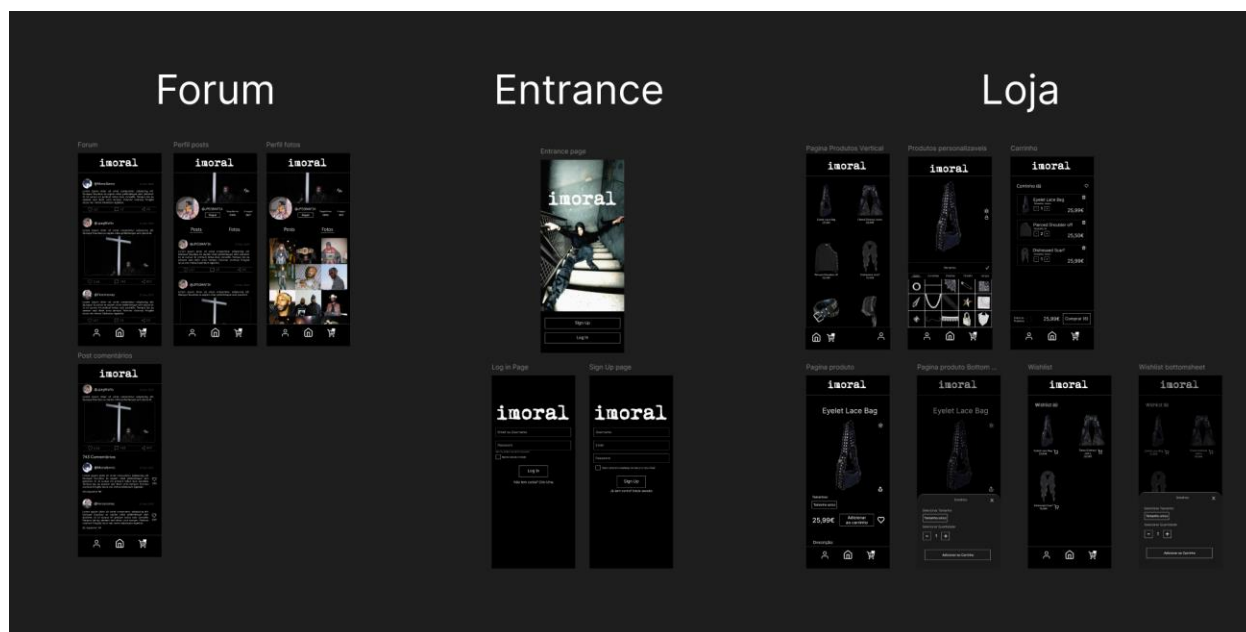


Fig.8 - Layouts feitos no Figma para a App Android

6.3 Implementação

A fase de implementação do projeto foi dividida em três partes principais: a base de dados, a aplicação móvel e o site administrativo. Para que todas estas componentes comunicassem bem entre si, foi desenvolvida uma API em PHP, com *tokens* JWT para autenticações seguras.

6.3.1 Base de Dados

Como o nosso sistema inclui uma loja e um fórum, foi necessária a criação de uma base de dados extensa e sólida, para que todos os dados se relacionem entre si sem qualquer conflito.

6.3.2 Aplicação Android

Após a construção da base de dados, foi necessário verificar quais as tabelas que precisavam de ser recriadas em Java para fazer parte dos *models* da aplicação. O design dos *layouts* no Figma também simplificou bastante a criação dos *layouts* em XML da aplicação. Com os *models* e os *layouts* criados, foi necessário então criar as *activities* que conectariam ambos de forma a tornar a aplicação interativa e funcional. Por fim, a última etapa da aplicação Android foi conectá-la à API do sistema realizada em PHP; para isto foi necessário implementar algumas *libraries* no Android Studio e também a criação de classes em Java que controlavam as chamadas HTTP da aplicação para a API.

6.3.3 Comunicação entre a App e a API: Exemplo Prático

Para exemplificar como funciona esta comunicação, vou usar novamente o exemplo do `getAllPosts`, mostrando agora o lado da aplicação Android. O método responsável por obter os posts do fórum faz um pedido GET à API. Primeiro, vai buscar o JWT guardado nas `SharedPreferences` para autenticar o pedido, e envia-o no header `Authorization`. Quando recebe a resposta, converte o JSON devolvido para um objeto

ForumResponse e, caso a operação tenha sido bem-sucedida, atualiza a lista do adapter com os posts recebidos, que o RecyclerView apresenta depois no ecrã.

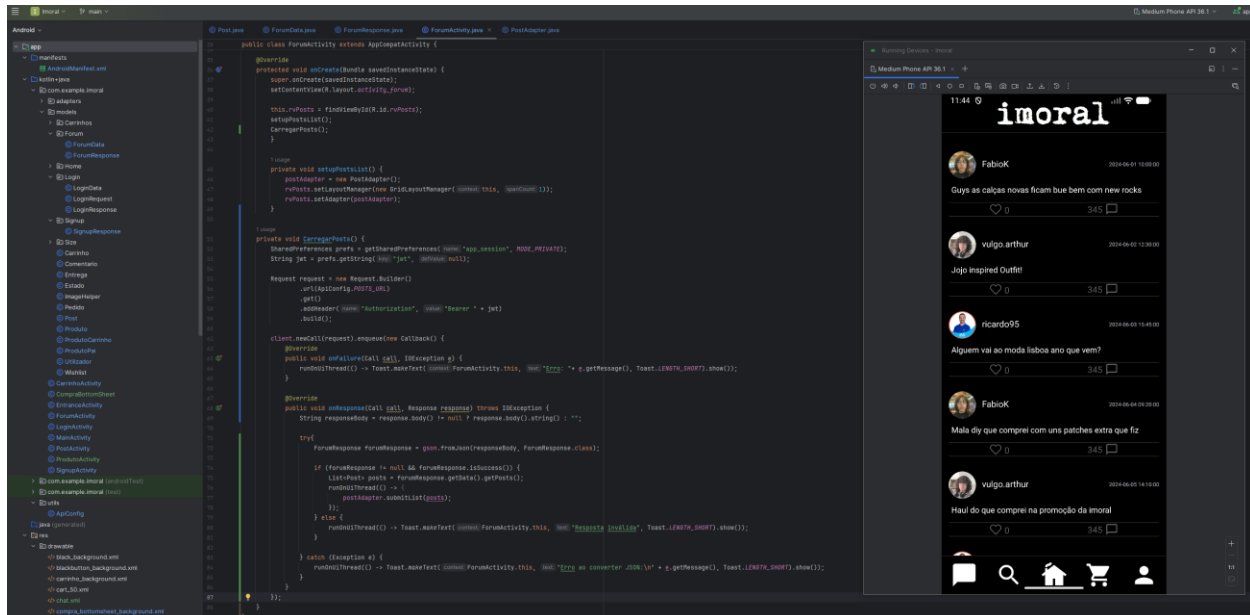


Fig.9 - Método CarregarPosts() na Classe ForumActivity e como a Activity fica na App

6.4 Testes e validações

Durante o desenvolvimento do projeto, todas as funcionalidades foram testadas antes de serem consideradas completamente implementadas.

Para testar os modelos em Java foi utilizado o BlueJ, que contém várias ferramentas que facilitam o teste de classes em Java.

Para testar a aplicação, fomos trabalhando gradualmente e testando cada parte que íamos implementando. O Logcat do Android Studio ajudou-nos a verificar o que causava os erros e o que acontecia por detrás da interface da aplicação.

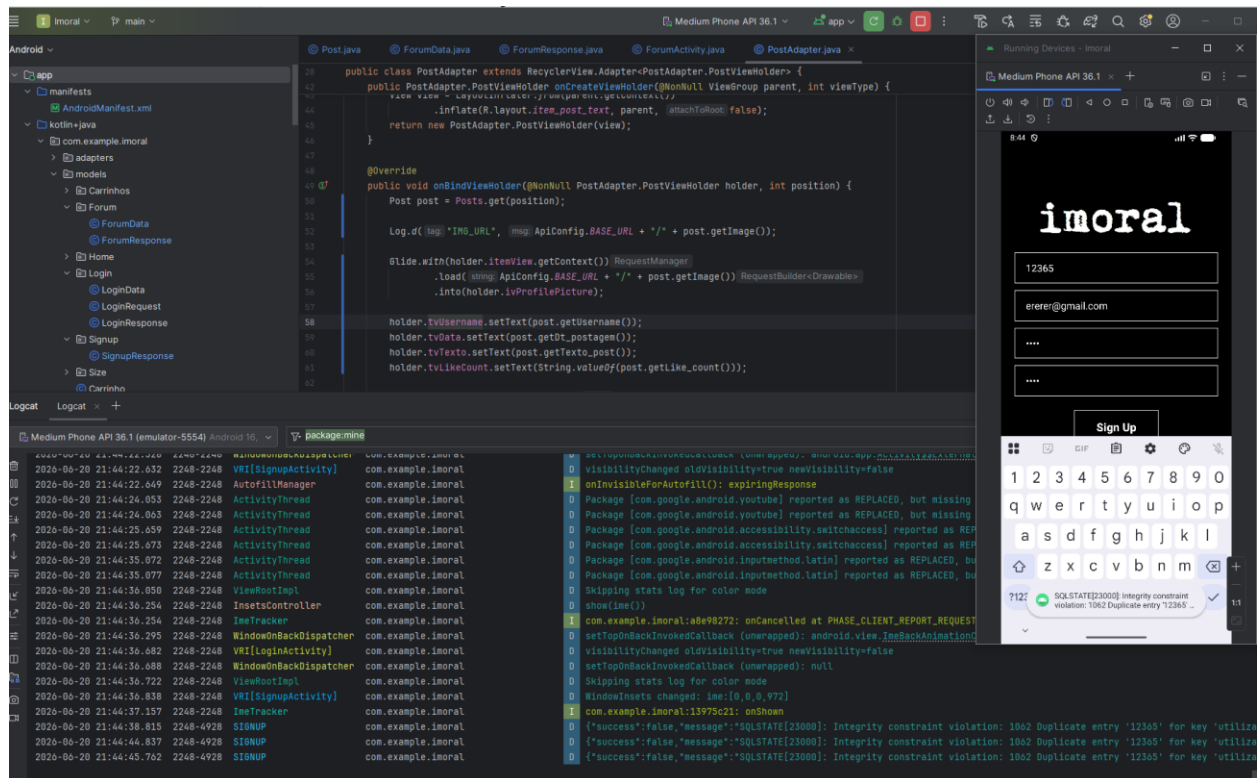


Fig.10 - Teste realizado pelo LogCat do AndroidStudio mostra o que a API retornou

Postman

Para testar as chamadas à API foi utilizado o Postman, uma ferramenta que permite enviar pedidos HTTP (GET, POST, PUT e DELETE) para os diferentes endpoints da API e analisar de forma simples e intuitiva o que estes retornam, facilitando a deteção de erros no código PHP.

Com o Postman foi possível verificar o código de estado (status code) devolvido por cada pedido, como por exemplo 200 (sucesso), 400 (pedido inválido), 401 (não autorizado) ou 500 (erro interno do servidor), o que ajudou a perceber rapidamente se um erro tinha origem no pedido enviado ou no próprio código da API. Foi também possível analisar o corpo da resposta (response body) em formato JSON, confirmando se os dados devolvidos correspondiam ao esperado, tanto em estrutura como em conteúdo.

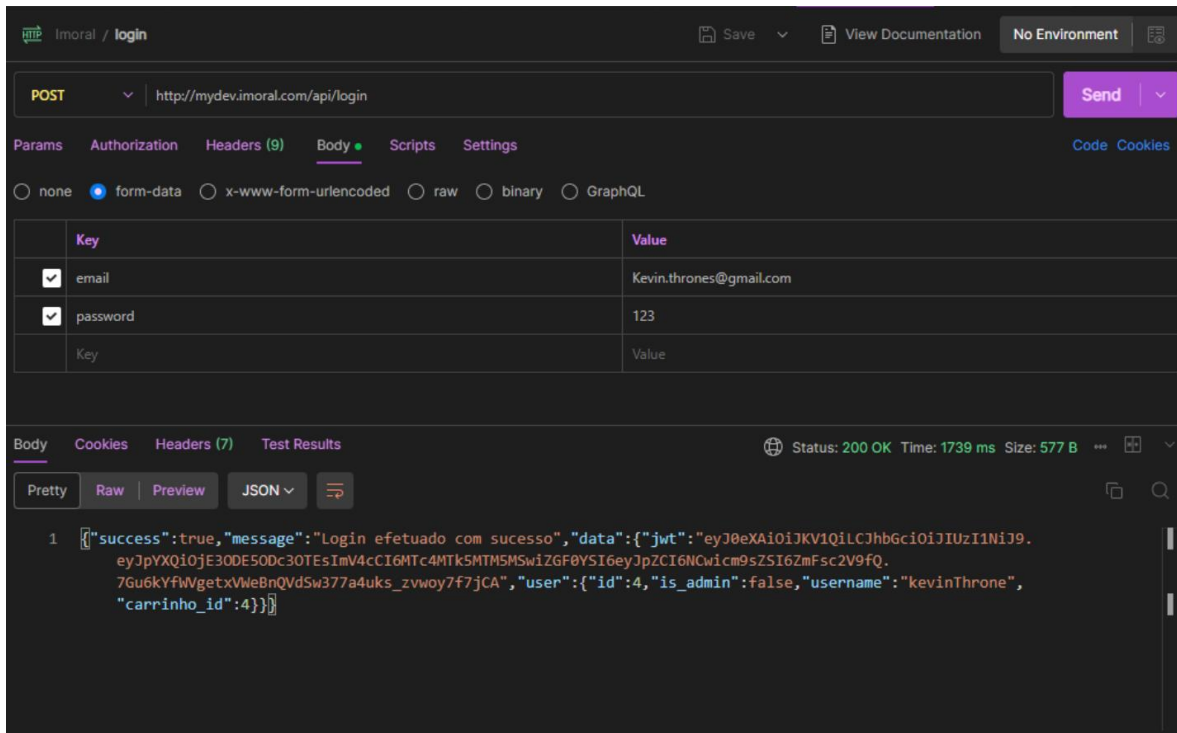


Fig.11 - Teste do LoginApi pelo Postman com retorno positivo

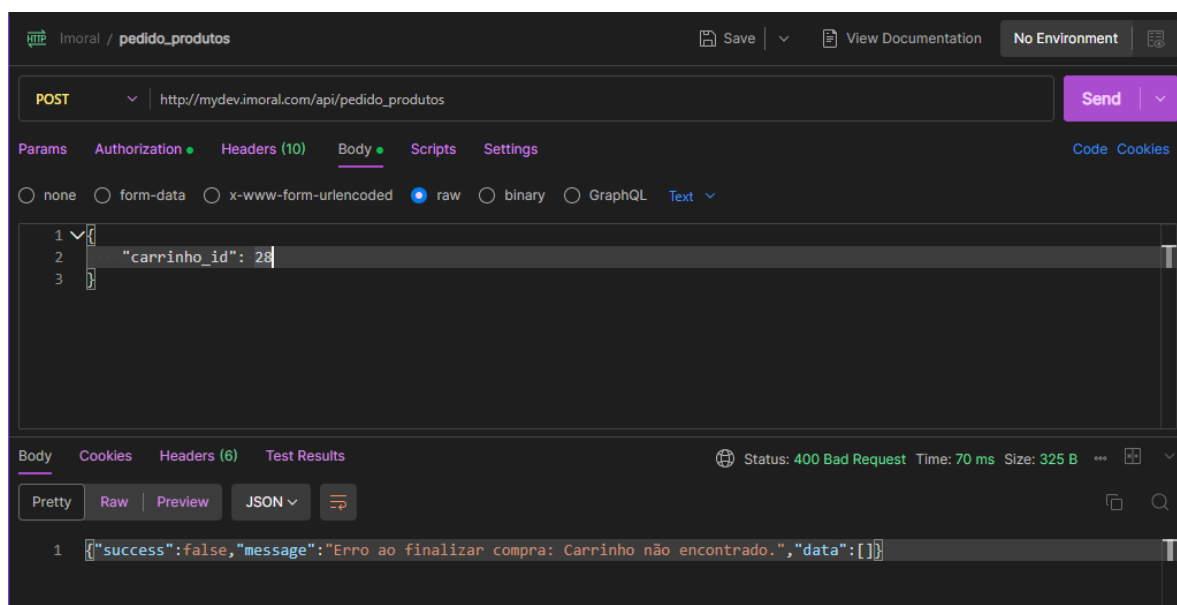


Fig.12 - Teste da rota pedido_produtos no postman com retorno negativo

7. Resultados

7.1 Funcionalidades Desenvolvidas

Ao longo do desenvolvimento do projeto Imoral foram implementadas as funcionalidades essenciais para o bom funcionamento de uma loja e de um fórum integrados na mesma aplicação, desde a gestão de conta até à compra de produtos e interação no fórum.

- **SignUp e Login:** O utilizador pode criar uma conta pela app, recebendo um email de confirmação, e, após a confirmação, pode fazer login com essa conta.
- **Visualização de Produtos:** O utilizador consegue ver todos os produtos pela app, assim como as suas imagens e detalhes, podendo escolher o tamanho e a quantidade que deseja comprar antes de adicionar ao carrinho.
- **Carrinho:** O utilizador pode adicionar, remover e alterar produtos no carrinho. O carrinho é visível pela app e mostra todos os detalhes dos produtos selecionados.
- **Compras:** O utilizador pode realizar compras pela app e acompanhar o estado das suas encomendas.
- **Fórum:** No fórum, o utilizador pode ver posts criados por outros utilizadores, fazer as suas próprias publicações, comentar e ver comentários nos posts, e ainda visitar perfis de outros utilizadores.

7.2 Benefícios Esperados

Esperamos que com este projeto a comunidade de moda alternativa em Portugal se fortaleça e que haja um maior incentivo no apoio a marcas independentes e na compra de roupa feita de forma sustentável.

7.3 Limitações

Apesar dos bons resultados obtidos e das funcionalidades já implementadas, o sistema ainda apresenta algumas limitações que devem ser consideradas, maioritariamente relacionadas com o tempo disponível e os recursos técnicos:

- O sistema está disponível apenas para dispositivos Android (sem versão para iOS);
- A base de dados está alojada localmente, o que impede o acesso online permanente;
- Não foi implementado um sistema de pagamento completo, nem exemplos para todos os métodos de pagamento;
- As funcionalidades de loja e fórum foram desenvolvidas apenas para a aplicação móvel, não existindo uma versão para site.

Estas limitações não comprometem o funcionamento do sistema, mas representam oportunidades claras de evolução futura.

8. Conclusão

8.1 Síntese do Projeto

Acredito que este projeto tem o potencial para atingir o seu objetivo e estou satisfeito com o que conseguimos fazer até agora. Tenho que admitir que o ritmo de trabalho podia ter sido mais fluido e, dessa forma, ter evitado que algumas tarefas ficassem para a última hora e que algumas funcionalidades não tivessem sido implementadas.

8.2 Reflexão Crítica

Este projeto ensinou-me a importância que uma boa metodologia de trabalho pode ter em projetos grandes. Planear e estruturar o que fazer antes de o começar é o ponto-chave de um projeto conciso e sólido. Todo este planeamento torna-se ainda mais essencial em projetos de grupo ou de dupla, como foi o caso deste projeto. Separar tarefas e verificar o trabalho feito pelo colega torna os objetivos do projeto mais alcançáveis, de forma mais rápida e eficiente.

8.3 Melhorias Futuras

No futuro planeamos ir melhorando a app com todas as funcionalidades que não conseguimos implementar no tempo que nos foi dado. Queremos melhorar o fórum com mais opções de publicações e de pesquisa por conteúdos, pretendemos tornar a aplicação compatível com iOS e passar todas as funcionalidades da app para um site acessível por computador.

9. Referencias

- **W3Schools.** (n.d.). Tutoriais de Desenvolvimento Web e Programação. Disponível em: <https://www.w3schools.com/>

Esta plataforma foi a minha principal referência para rever sintaxe, testar pequenos exemplos de código e esclarecer conceitos relacionados com HTML, CSS, JavaScript, PHP, SQL e MySQL ao longo do desenvolvimento do projeto.

- **Stack Overflow.** (n.d.). Community Help and Code Examples. Disponível em:

<https://stackoverflow.com/>

Recorri a este fórum sempre que surgiam erros difíceis de resolver sozinho, encontrando discussões de outros programadores que ajudaram a perceber a origem dos problemas e possíveis soluções.

- **YouTube.** Tutoriais sobre criação de APIs RESTful, PHP e desenvolvimento Android – Augusto Gonçalves. Disponível em: <https://www.youtube.com/>

Também assisti a explicações passo a passo, feitas pelo Professor José Augusto da Rocha Gonçalves, sobre como construir uma API RESTful e como implementá-la no android Studio. (Os vídeos individuais não foram listados por serem vários.)